codecademy

Models and Databases

MTV

In Model-Template-View paradigm, the model determines the structure and type of data, the view is the data to be presented, and the template is how the data is presented.

Models

The model is the single source of information containing essential fields and behavior of data.

```
from django.db import models

class Person(models.Model):
  first_name =
models.CharField(max_length=30)
  last_name =
models.CharField(max_length=30)
```

Database

Django provides database storage that can be used with the Django programming interface.

Default Database

Django will by default create an SQLite database in the file **db.sqlite3**.

```
└── myproject/
  └── myproject/
  └── manage.py
  └── db.sqlite3
```

Mapping Models to Databases

Each model attribute maps to a column in a relational database table. In the provided example, see the raw SQL created that Django translated from a `Car` model.

```
CREATE TABLE myapp_car (
  id integer NOT NULL PRIMARY KEY
AUTOINCREMENT,
  make varchar(20) NOT NULL,
  model varchar(20) NOT NULL,
  year date NOT NULL
);
```

models.py

Models are defined in **models.py** within the app's folder.

```
from django.db import models

class Car(models.Model):
  make = models.CharField(max_length=20)
  model = models.CharField(max_length=20)
  year = models.DateField()
```

Primary Keys

A primary key is a column that uniquley identifies each row in a relational database.

```
class Car(models.Model):
  make = models.CharField(max_length=100,
primary_key=True)
```

Foreign Keys

A `ForeignKey` field type setups up a one-to-many relationship between models.

```
class Author(models.Model):
  name = models.CharField(max_length=64)

# Book has a many-to-one relationship with Author
class Book(models.Model):
  isbn = models.CharField(max_length=15, primary_key=True)
  title = models.CharField(max_length=255)
  author = models.ForeignKey(Author, on_delete=models.CASCADE)
```

Model Methods

Models come with predefined methods that are inherited from the `Model` parent class. These models are invoked on the model instances, not the model class.

```
model_instance.save()
```

Model Metadata

Every model can be defined with optional metadata, which is anything that is not a field.

```
class Person(models.Model):
  age = models.IntegerField()

  class Meta:
    ordering = ["age"]
    verbose_name_plural = "people"
```

Verbose Names

Every model class has a metadata option, `verbose_name`, which serves as a human-readable class name.

```
first_name = models.CharField("person's first name", max_length=30)
```

Field Arguments

Default values can be added as optional arguments in models.

```
from django.db import models
class Applicant(models.Model):
  GRADE = [
    ('P', 'Pass'),
    ('F', 'Fail')
  ]
  grade = models.CharField(max_length=1, choices=GRADE, default='P') "
```

Choices in Models

The `choices` key can be used by providing an array of tuples where the first value is the actual value to be set in the database, and the second value is a human-readable name.

```
from django.db import models
class Applicant(models.Model):
  SINGLE = 'S'
  MARRIED = 'M'
  DIVORCED = 'D'
  STATUS = [
    (SINGLE, 'Single'),
    (MARRIED, 'Married'),
    (DIVORCED, 'Divorced')
   ]
  marital_status = models.CharField(max_length=1, choices=STATUS, default=SINGLE)
```

Migrations Process

There are two consecutive steps to perform migrations. First, the `makemigrations` command will commit model changes into migration files. Then the `migrate` command can be used to apply changes to the database schema.

```
python3 manage.py makemigrations
python3 manage.py migrate
```

makemigrations

The `makemigrations` command packages any model schema changes inside **models.py** into migration files. The name of an app is optional but is helpful when there are multiple apps in a project.

```
python3 manage.py makemigrations
```

Migration Files

Migration files are created in the **migrations** directory.

```
└── myproject/
  └── migrations/
```

Applying Migrations

Migrations can be applied to an existing database schema and an app name can be provided to target a specific database schema.

```
python3 manage.py migrate myapp
```

Rolling Back Migrations

Migrations can be rolled back to a specific migration using the `migrate` command, the name of the app, and the name of the migration file to revert back to. In the example, the app will revert back to the `0001` migration.

```
python3 manage.py migrate app_name 0001
```

CRUD

Basic database operations are commonly referred to as CRUD- Create, Read, Update, and Delete.

Structured Query Language

The underlying language for CRUD operations is Structured Query Language, SQL.

Python Shell

CRUD operations can be executed in the Python shell.

Importing Models

Models must be imported into the Python shell to be used.

```
from app_name.models import ModelName
```

Model Instance

An instance of a model can be created by calling the model with the fields as arguments.

```
model_instance = ModelName(field1="field 1 value")
```

.save() Method

A model instance can be saved to the database using the `.save()` method.

```
model_instance.save()
```

Viewing All Instances

All the instances of a model can be returned as a QuerySet using the `.all()` method.

```
model_instance = ModelName.objects.all()
```

QuerySet

A QuerySet is a collection of objects from a database.

QuerySet Index

A QuerySet object can be retrieved using bracket notation.

```
c = Client.objects.all()
obj = c[1]
```

.first() Method

The first instance of a model can be retrieved using the `.first()` query method.

```
first_instance = ModelName.objects.first()
```

Model Field

A model instance's field value can be accessed using dot notation.

```
mode_instance.field_name
```

Updating Field Value

A field's value can be reassigned using dot notation.

```
mode_instance.field_name = "New value"
```

Save Model Instance

An updated model instance must be saved into the database using the `.save()` method.

```
first_instance.save()
```

Reverse Relationships

The reverse relationship can be queried using the `._set` property, where the `_set` property is preceded by the lowercase name of the model.

```
question_instance.answer_set.all()
```

Deleting Model Instance

Model instances can be deleted using the `.delete()` method.

```
first_instance.delete()
```

Implementing .CASCADE

`.CASCADE` must be implemented on the model itself and will delete everything related to the instance of that model.

```
model_instance =
models.ForeignKey(ModelType, null=True,
on_delete=models.CASCADE)
```

.get() Method

The `.get()` method returns a single object that matches the arguments provided. If `.get()` matches multiple objects, a `.MultipleObjectsReturned` exception will return.

```
skate_blog =
Blog.objects.get(name="Skating Post")
```

.get_or_create() Method

The `.get_or_create()` method will return a tuple that contains an object, with the supplied arguments, and a boolean that states whether the object was created or not.

```
(<ModelName: ModelName object (15)>, True)
```

.exclude() Method

The `.exclude()` method returns all objects that do not match provided the arguments.

```
ModelName.objects.exclude(FieldToExclude="
Value")
```

.order_by() Method

The `.order_by()` method returns a list of objects based on a specified order which is provided as an argument.

```
Entry.objects.order_by('id')
```

Foreign Keys

Foreign Keys can connect two tables together through a one-to-many relationship.

```
class Book (models.Model):
  author =
models.ForeignKey('Author',on_delete=model
s.CASCADE)
```

.filter() Method

The `.filter()` method is used to search for a model instance related to another model instance.

```
Answer.objects.filter(question_id=3)
```